



Resumen pedagógico: Cómo crear un Pull Request en GitHub

Introducción rápida

- El video, presentado por Eduardo, explica paso a paso qué es un Pull Request y cómo hacer uno en GitHub usando un ejemplo práctico.
- Idea central: un Pull Request (PR) es una propuesta de cambio que tú, como contribuidor, envías a un proyecto en el que no eres propietario. Los mantenedores pueden aceptar o rechazar esa propuesta.

Pasos detallados para hacer un Pull Request (explicado paso a paso)

1. Elegir el repositorio objetivo

- Escoge el proyecto al que quieras aportar (puede ser una web, API, librería, framework, etc.).
- En el ejemplo del video el repositorio es muy simple y solo tiene un README con una lista de colaboradores.

2. Hacer fork del repositorio

- Si no eres administrador del repositorio, primero haz un fork (botón “Fork”) para crear una copia bajo tu cuenta.
- Esto te permite trabajar sin tocar directamente el proyecto original (ideal para colaborar desde Colombia o cualquier lugar sin permisos).

3. Clonar tu fork a tu máquina

- En tu terminal: `git clone`
- Ingresa a la carpeta clonada para trabajar localmente.

4. Crear una rama (branch) nueva

- Nunca trabajes directamente sobre main. Crea una rama para tu cambio:
 - `git checkout -b nombre-de-la-rama` (por ejemplo: apellidos)
- Esto mantiene tu trabajo aislado y facilita la revisión.

5. Hacer los cambios en la rama

- Edita los archivos que necesites. En el ejemplo, Eduardo modifica README.md para añadir sus apellidos.
- Revisa los cambios localmente.

6. Registrar los cambios (commit)

- Agrega y commitea:
 - `git add .`
 - `git commit -m "mensaje descriptivo"` (ej.: "nuevos apellidos")
- Los commits deben explicar qué cambiaste para que los revisores lo entiendan.

7. Enviar la rama al repositorio remoto (push)

- Para que el fork en GitHub tenga tu rama: `git push origin nombre-de-la-rama`
- Ahora tu rama y tus cambios estarán disponibles en tu repositorio remoto (tu fork).

8. Abrir el Pull Request desde GitHub

- En el fork, GitHub suele mostrar un botón “Contribute” o “Compare & pull request”.
- Rellena el título y la descripción del PR explicando los cambios (ej.: "Agregar apellidos del autor").
- Envía (Create Pull Request). Esto propone integrar tus cambios al repositorio original.

9. Revisión y decisión por los mantenedores

- Los administradores del proyecto original reciben notificación (correo) y ven el listado de PRs en la pestaña “Pull requests”.
- Ellos revisan los archivos modificados y pueden aceptar (merge) o rechazar.
- Si hay conflictos, deben resolverse antes de hacer el merge.

10. Aceptar/mergear el PR (si corresponde)

- En el ejemplo, el autor del repositorio acepta el PR y hace merge.
- Resultado: los cambios (p. ej. “Eduardo Ismael García Pérez”) quedan integrados en el repositorio principal.

Buenas prácticas y consejos (estilo práctico, al estilo colombiano)

- Usa ramas con nombres claros: feature/apellidos, fix/errores, docs/README.
- Mensajes de commit descriptivos: ayudan a tus compas y a los mantenedores.
- Si vas a proponer cambios pequeños (correcciones ortográficas, agregar tu nombre), igual crea una rama: eso simplifica la revisión.
- Participar en proyectos que te gusten te puede abrir puertas: tu aporte puede ser aceptado y quedar en el historial del proyecto.
- Recuerda: si te rechazan, no pasa nada. Aprendes y mejoras tus contribuciones.

Resumen final

- Un Pull Request es la forma estándar de proponer cambios en proyectos de GitHub cuando no eres propietario.
- Flujo esencial: fork → clone → rama nueva → cambiar → commit → push → abrir PR → revisión → merge/rechazo.
- El video muestra todo el proceso con un ejemplo práctico y accesible, ideal para quien quiera empezar a colaborar en proyectos open source desde Colombia o cualquier otro lugar.

Si quieres, te puedo dar un checklist listo para seguir cuando vayas a crear tu primer PR. ¿Te interesa?

Errores Comunes:

- **Errores de sintaxis:**
 - Falta de paréntesis, comillas o dos puntos.
 - Errores de indentación.
 - Uso incorrecto de operadores.
- **Errores lógicos:**
 - Condiciones incorrectas en las sentencias if.
 - Bucles infinitos.
 - Cálculos incorrectos.
 - Variables no inicializadas.
- **Errores de nombres:**
 - Variables o funciones con nombres poco descriptivos.
 - Nombres de variables que no siguen las convenciones del lenguaje (e.g., camelCase para variables, PascalCase para clases).